Development of a Deep-Learning System for Automated Tree Recognition & Green-Space Mapping in Urban Environments

Rasul Aidarkhanov · Berik Sharipov · Anuar Totin

Supervisor: Syndar Satbayev

Astana IT University · 6B06101 Computer Science · 2026

Contents

Relevance

Urban trees regulate microclimate, sequester carbon and mitigate the heat-island effect — so a growing city like Astana needs an **up-to-date, spatially accurate inventory** of its green infrastructure for planning and climate-adaptation decisions.

Traditional inventories rely on **manual field surveys** — slow, expensive, and outdated before completion. A fundamental scalability problem for any large urban area.

Deep-learning detection on freely-available satellite imagery is a feasible alternative — but the real gap is that **no automated process for this exists in Astana at all.** So the task we set ourselves was an engineering one: **build that process.**

Why we couldn't just use an existing tool

0.012

Box mAP@50 — a popular ready-made detector (NEON DeepForest), run as-is on Astana. Essentially **blind.** There was no off-the-shelf system that worked here, so building one was the only option.

Research aim

Design, build and evaluate an **automated end-to-end system** that recognises trees and maps green space from Astana satellite imagery — delivered as a working tool, with detection accuracy reported honestly.

The thesis, in one idea

The deliverable is a system — not a model

Our title says **"automated … system."** So the unit of work isn't "which neural net scores highest" — it's the whole machine that turns a raw satellite capture into a usable, georeferenced tree inventory. Everything else is a **component of building that:**

Dataset
What the engine learns from — a means, not the goal.

The models
Four interchangeable engines that plug into the pipeline.

The experiments
How we tuned the engine — evidence of depth, not the product.

UI & features
What makes it usable by a non-technical operator.

We could have shipped the bare minimum — one model, a mask, no metrics. Instead we built a **tool someone at a city service could actually operate.** The slides that follow lead with the system; the models come later, as parts of it.

Objectives & methods

1. Build an **annotated Astana dataset** (tiled satellite imagery, instance polygons).
2. Train and compare four **detection engines** — YOLOv8-seg, Mask R-CNN, DeepForest, SAM 2.
3. **Combine** them through ensembles and test whether that helps.
4. Evaluate everything on **one common test set** for a fair comparison.
5. Wrap it all in an **automated web application** — the deliverable that ties tasks 1–4 together.

Methods

Supervised deep learning · transfer learning from pretrained weights · sliding-window tiling · pixel → WGS-84 geo-referencing · mAP@50 as the single comparison metric · software engineering (FastAPI · React · SQLite).

Problem statement — Input / Output

Input

A satellite capture of an area of Astana at zoom 19 (≈ 0.3 m/px) — uploaded, or drawn on a map and auto-fetched from **ESRI or Google** tiles — cut into overlapping **640 px RGB tiles**. Plus the operator's choices: **model, confidence, geo-mode.**

Output

For every detected tree: a **bounding box**, an **instance mask**, a **confidence score**, **crown area** and **lat/lng**. Tiles are stitched, duplicates merged, results shown on a **map** with per-area counts and **GeoJSON / CSV / HTML** export.

capture → tile → model $f(\cdot)$ → $\{$ box, mask, conf, area, lat/lng $\}_{\text{per tree}}$ → merge & map & export

Literature review — what others report, and the gap each leaves

Method	Authors · year	Data	Best metric	Gap for our problem
Mask R-CNN (MCAN)	Lv et al. · 2023	UAV RGB, China	**Det AP 92.4%**	*Forest UAV, not urban satellite; research result only*
RetinaNet / Faster R-CNN	dos Santos et al. · 2019	UAV RGB, Brazil	**AP 82–93%**	*Detection only — no segmentation, no system*
YOLOv12m	Abbas & Damaševičius · 2025	RGB satellite	**mAP@50 90.8%**	*Benchmark on a clean public set; no deployable tool*
DeepLabV3+ / U-Net	Martins · 2021 / Wang · 2021	Aerial 10–32 cm	**F1 91% · IoU 96%**	*Semantic mask, not per-tree instances; not Central Asia*
DeepForest (off→fine)	Ventura et al. · 2024	NAIP 60 cm, USA	**F 0.42 → 0.73**	*US imagery; collapses without local fine-tuning*
YOLOv5x	Velasquez-Camacho · 2023	Ground-level, Spain	**F1 84.9%**	*Europe, ground-level; no GIS/coordinate pipeline*
DeepForest (urban)	Dakov & Petrova-Antonova · 2024	Aerial, Sofia	**F1 0.67–0.69**	*Closest analogue — but still Europe, still research-only*

Every study reports high accuracy — on **its own region and sensor**, *as a* **research result**. *Overall limitation:* **none was tested on Central Asia, and none delivers a deployable, operator-run system.** *That gap — not a higher mAP — is what this project fills. (31 references reviewed.)*

The core of the project

The automated pipeline

Front end — React 18 + Leaflet

API — FastAPI

Sliding-window tiling (640 + 128)

Engine — swappable (YOLO · M-RCNN · DF · Ensemble)

Global NMS — cross-tile merge

Geo — pixel → WGS-84

SQLite store → Map & export

One automated chain

Capture → tile → detect → merge → georeference → store → map → export. **No manual step** between the operator drawing an area and the finished, downloadable inventory.

Swappable engine — the key design idea

The detection model is just one plug-in step. Swap YOLO for Mask R-CNN, DeepForest or an ensemble without touching the rest — a better model tomorrow drops straight in.

Performance

~1 km² at zoom 19 in ≈ **18 s** on a single laptop GPU, with live streaming progress.

Engineering effort

A minimal version would do far less — we built a tool

THE BARE MINIMUM

- One model, hard-coded
- Just a segmentation mask
- No coordinates, no export
- No metrics, no comparison
- Run once, nothing saved

WHAT CANOPY ACTUALLY DOES

+ **Two tile providers** — ESRI & Google, fetched in-browser
+ **Model choice** — 7 YOLO sizes · Mask R-CNN · DeepForest ± SAM 2
+ **Ensembles** — Weighted Box Fusion + cross-YOLO vote
+ **Manage scans** — name · hide · delete (cascade)
+ **Live metrics** — tree count · canopy coverage % · crown area
+ **Geo + export** — WGS-84 · GeoJSON / CSV / HTML · persistent map

None of these is required to "detect a tree" — each is a deliberate choice to make the system **usable, comparable and repeatable.** *That is where most of the engineering effort went.*

A component of the system — inputs & infrastructure

Data, database & hardware

Dataset

High-zoom Astana captures (~0.3 m/px). We hand-labelled **~5 500 tree crowns** across **~100 images** as instance polygons (≈ 8 700 training tiles after 640+128 tiling). A held-out common set of **14 images · 702 trees** scores every model.

Database — SQLite

scan_sessions → snapshots → runs → detections

One-to-many, **ON DELETE CASCADE** — delete a scan, its detections go too.

Hardware

RTX 4060 8 GB — YOLO + all common-set inference · RTX 4070 — Mask R-CNN · RTX 4050 — DeepForest. The 8 GB ceiling forces batch 2 + mixed precision — deliberately modest, to prove it runs **without a cluster.**

The maths, in plain terms

Three simple ideas — one comparison number

1 · IoU — did the box land on the tree?

IoU = (overlap area) ÷ (combined area). **1.0** = perfect, **0** = miss. A tree counts as found when **IoU ≥ 0.5.**

2 · Precision & recall

Precision — of the trees we flagged, how many were real. **Recall** — of all real trees, how many we found: **ours ≈ 30%** at the default setting.

3 · mAP@50 — the single score

Sweep the confidence threshold, track precision against recall, take the **area under that curve** (at IoU ≥ 0.5). One honest number to compare every engine the same way — the **0.315** on the results slide.

How the two detectors work

YOLOv8 — one pass: grid → boxes + masks in a single look (fast). **Mask R-CNN** — two passes: propose regions, then refine (slower, careful).

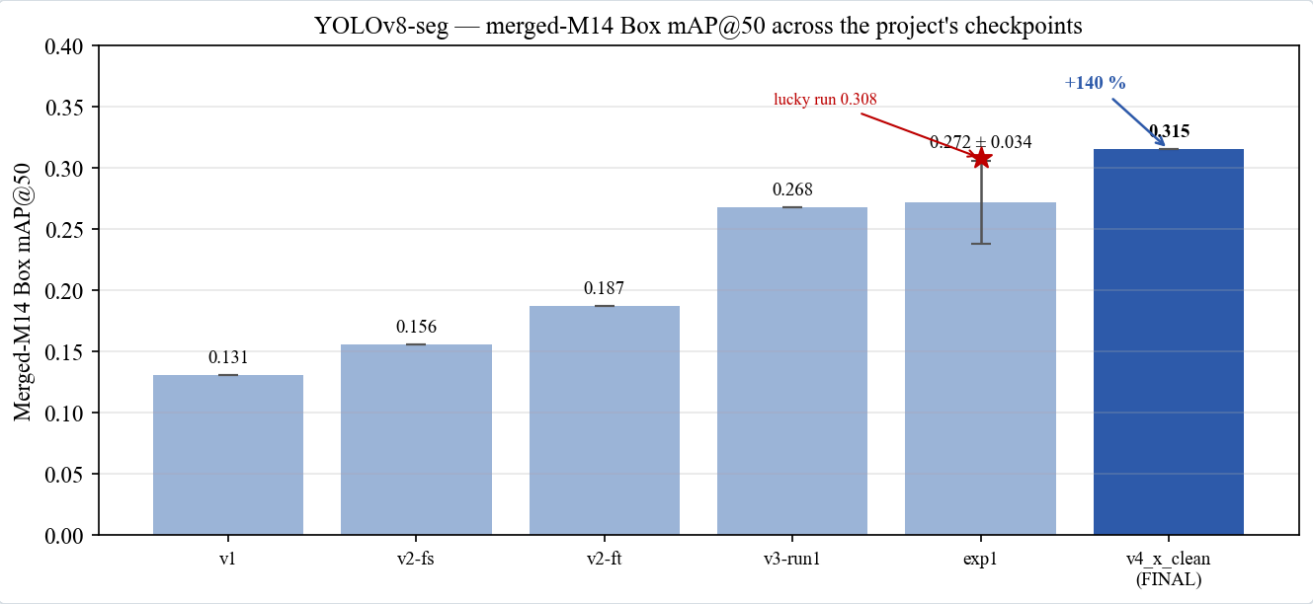
Four interchangeable engines · one pipeline

The system doesn't depend on one model — any of four can drive it.

YOLOv8-seg (Rasul) · Mask R-CNN (Berik) · DeepForest + SAM 2 (Anuar) · plus two ensembles. Each plugs into the same automated pipeline and is scored on the same 14 images.

Engine #1 · Rasul Aidarkhanov · 23 experiments

YOLOv8x-seg — from 0.131 to 0.315



Box mAP@50 across every experiment on the common set.

+140%

Box mAP@50 over my own first model (0.131 → **0.315**).

What actually mattered

① Start from pretrained (COCO) weights · ② tile at the resolution you deploy at · ③ keep augmentation moderate.

What I also found

Default augmentation beat hand-tuned; model size is **U-shaped**; run-to-run noise ≈ ±0.03 — so small gaps aren't real.

Engine #2 · Berik Sharipov

Mask R-CNN — the careful, mask-first engine

A classic **two-stage, region-based** detector. Where YOLO looks once, Mask R-CNN looks twice — propose, then refine — slower, but with naturally **crisp instance masks.** One of the swappable engines behind the same automated pipeline.

How it works

An RPN suggests candidate boxes → RoIAlign crops each region → three heads output a box, a class and a per-pixel mask. Segmentation is built in, so outlines hug the crown.

How it does on Astana

Box mAP@50 **0.166**, Mask **0.158**. At conf 0.5: precision 0.44 · recall 0.22. Strongest on medium, well-separated crowns.

How Berik trained it

Transfer-learned from a COCO-pretrained ResNet-50-FPN, fine-tuned on the same Astana tiles (v2 + v3), single "tree" class, same split as every engine. 50 epochs, SGD.

Why it stays in the app

Below YOLO overall — but it recovers some **larger trees the others miss**, and its masks are the cleanest. So the pipeline keeps it as a selectable engine.

Engine #3 · Anuar Totin

DeepForest + SAM 2 — the forest specialist, re-taught

A two-part engine: **DeepForest** (a RetinaNet-style crown detector) finds the trees, then **SAM 2** turns each box into a high-quality mask.
It produced the **0.012** that started the whole project.

How it works

DeepForest outputs a box per crown; each box becomes a **prompt** for SAM 2, which segments the exact pixels — zero-shot, no extra training.

How Anuar fixed it

Fine-tuned DeepForest on the Astana data (LR 1e-4, 30 ep) + SAM 2 masks. Result: **0.012 → 0.146** Box mAP@50 (Mask 0.134) — a **×12** jump.

The domain problem

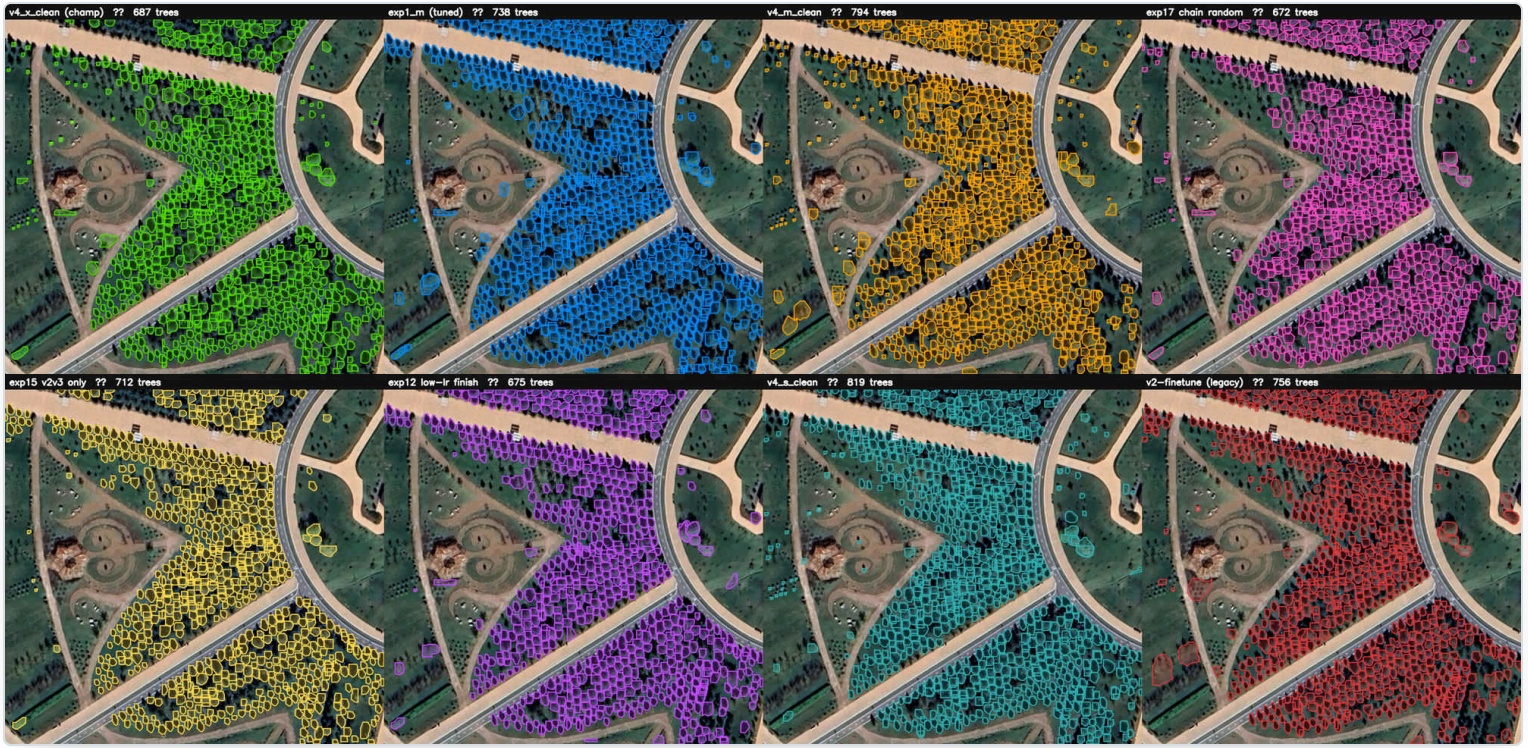
DeepForest is pretrained on **North-American forest** imagery. Pointed at a dry steppe city it was nearly blind — **0.012.** A mismatch, not a bug.

Why it stays in the app

SAM 2 gives the **best general-purpose masks** of any engine, and the off-the-shelf → fine-tuned story is the project's core lesson.

A system choice, not a model choice

Same garden, eight engines, eight answers



The same Botanical Garden, scanned by **eight engines** — each outlines a **different set of trees** and reports a different count (**672–819**). None is simply "right".

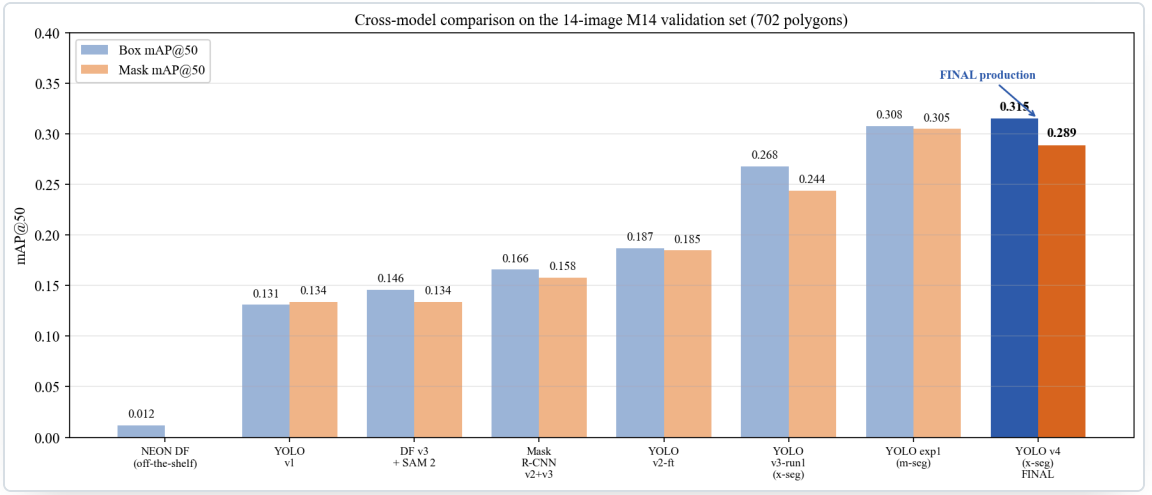
So the system lets the operator **switch engines** and combine them — **Weighted Box Fusion** + a **cross-YOLO 4-vote** that drops single-model false positives. Choice is a feature.

Head-to-head · same 14 images · 702 trees

One common test set, every model

Model	Box mAP@50	Mask mAP@50
YOLOv8x-seg v4 (ours)	**0.315**	**0.289**
YOLOv8x-seg v3	0.268	0.244
Mask R-CNN	0.166	0.158
DeepForest + SAM 2	0.146	0.134
YOLO v1 baseline	0.131	0.134
NEON DeepForest (off-the-shelf)	0.012	—

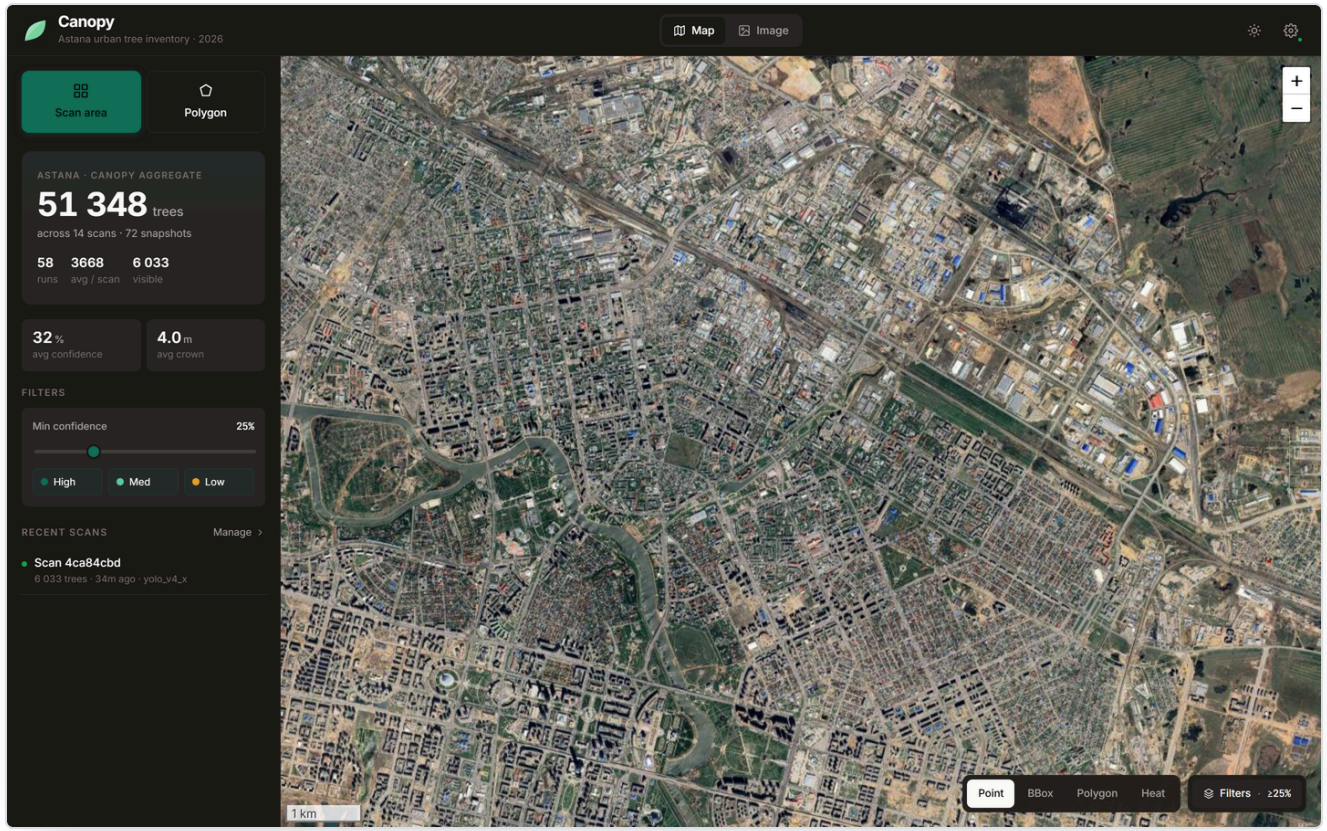
Champion operating point (conf 0.25): **precision 0.52 · recall 0.29.** *Margins between top models are small — partly within run-to-run noise.*



We read this as "these engines are in the same ballpark on Astana" — what matters is that ***any of them can drive the system.***

The deliverable

Canopy — the system in use



One flow

Select an area of Astana → the app tiles it, runs the chosen model, and draws every detected tree on the map — confidence filter, Point / Box / Polygon / Heat layers.

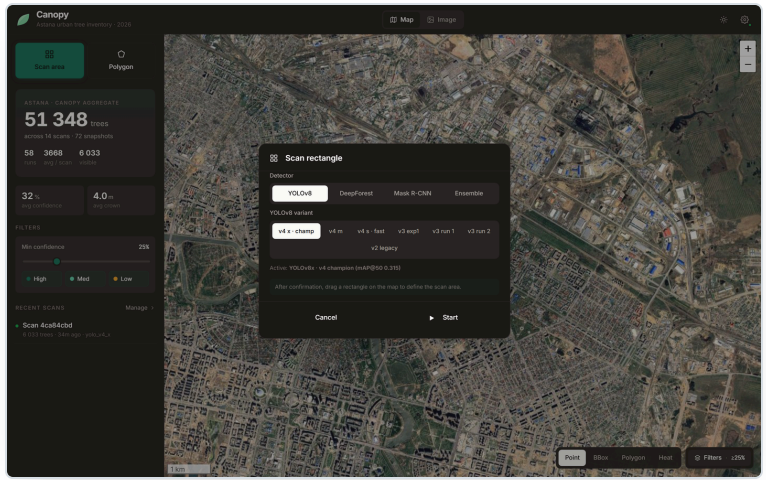
Grows as you use it

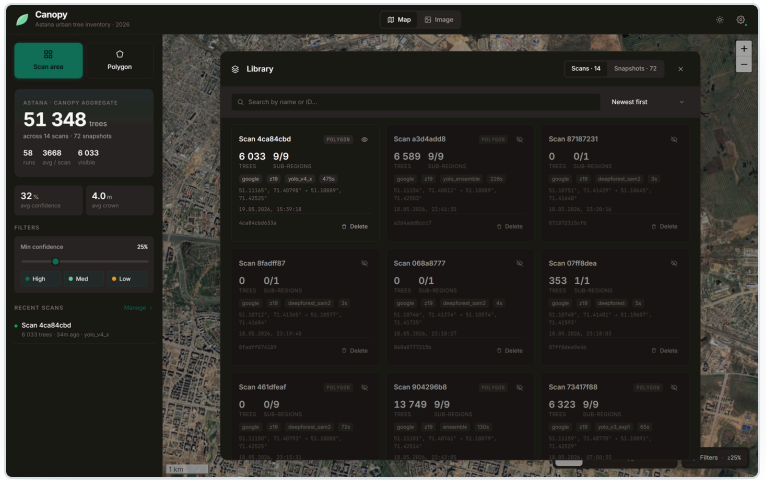
Every scan is stored; the city map is the running aggregate of everything ever processed. Coverage and counts update live.

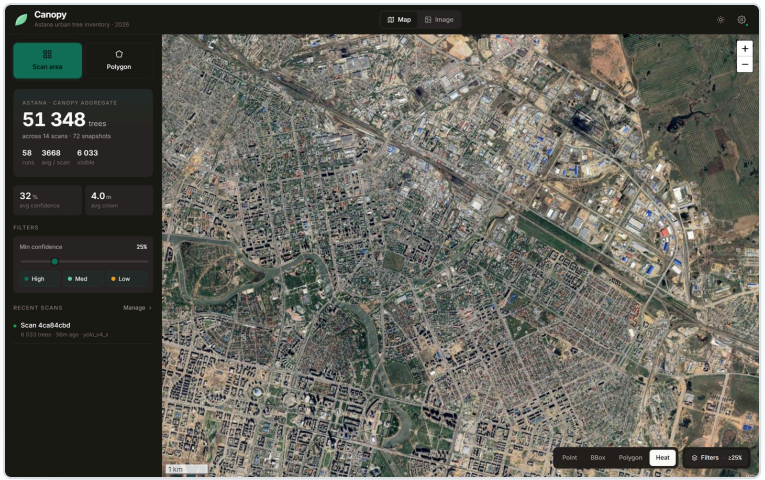
*Counts in screenshots come from repeated demo scans — **not** a full city census.*

The choices that make it a tool, not a demo

Choose · combine · manage · measure







Choose & combine

7 YOLO sizes · Mask R-CNN · DeepForest ± SAM 2 · WBF & cross-YOLO vote — chosen per scan, with provider (ESRI / Google) and confidence.

Manage

Every scan & snapshot is renamable, hideable and deletable (cascade) from a library modal — the workspace stays usable over time.

Measure

Density heat-map, per-area count, **green-space coverage %** and crown area inside a drawn polygon — metrics, not just dots.

The pipeline running end-to-end

Live demo — automated scan of the Botanical Garden

What you're seeing

The v4 engine scanning the Botanical Garden end-to-end in Canopy: **tile → detect → map**, every tree drawn live.

Why this scene

A dense, well-defined planting — the clearest case. Honest reminder: on sparser scenes recall drops.

Full recording

drive.google.com/file/d/1OceaLC4yMEgLtIThVR8t7_B4VRRPNaNf/view

Conclusion

O1 · Dataset

Custom Astana dataset built from scratch — ~100 images, ~5 500 crown polygons (≈ 8 700 tiles); held-out M14 = 14 images / 702 trees.

O2 · Detection engines

Four engines trained & compared; YOLOv8x-seg leads at Box mAP@50 = **0.315** (+140% over our v1 baseline).

O3 · Ensembles

WBF + cross-YOLO vote implemented in the system — choice and combination exposed to the operator.

O4 · Fair evaluation

Every engine scored on one common set — the **first such measurement for Astana** (off-the-shelf ≈ 0.012).

O5 · The system — Canopy

FastAPI + React + SQLite + Leaflet, deployed: provider & model choice, ensembles, manage, live metrics, geo + export.

The headline

Not a score — a **manual task is now an automated system.** Accuracy is modest and honestly so; the **system is the contribution.**

Thank you — we'll gladly answer your questions.

From a manual, one-by-one task to an **automated** tree-mapping system for Astana — a working tool anyone can run.

Rasul Aidarkhanov · Berik Sharipov · Anuar Totin | Supervisor: S. Satbayev